

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31
- 32
- 33
- 34
- 35
- 36
- 37
- 38
- 39
- 40
- 41
- 42
- 43
- 44
- 45
- 46
- 47
- 48
- 49

Additional Key Words and Phrases: LLM agents, Model Context Protocol, tool surface, least privilege, configuration analysis, information flow, prompt injection, empirical study

Anonymous Author(s). 2026. INTERLOCK. 1, 1 (September 2026), 18 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

에이전트가 외부 도구를 받아오는 표준은 MCP(Model Context Protocol)다 [13]. 사용자는 설정 파일에 서버를 적고, 서버마다 그것을 실행할 명령, 서버가 받을 자격증명, 그리고 (일부 서버는) 어떤 도구를 광고할지 정하는 플러그를 함께 적는다. 그러면 호스트가 서버를 띄우고 도구 목록을 받아, 각 도구의 이름과 설명, 스키마를 모델에게 보여준다. 이렇게 한 세션에 모인 도구 전부를 그 배포의 *tool surface*, 곧 모델이 부를 수 있는 도구의 집합이라 부른다. 이 방식은 이미 굳어졌다. 실제 애플리케이션 1,723개 중 85.2%가 서버 목록을 파일로 두는 반면, 도구 호출을 멈추고 사용자 승인을 받는 차단형 게이트를 두는 것은 37.2%뿐이다 [28]. 서버 한 줄을 적으면, 대개 그 서버의 도구는 모델이 부르기만 하면 실행된다.

실제 배포에서는 그 전체가 성립하지 않는다. 그림 1은 우리가 다루는 상황으로 MCP를 이용하여 자기 저장소의 열린 이슈를 정리하는 과정을 보여준다. 먼저 ①에서 개발자가 코딩 에이전트에 GitHub MCP 서버 하나를 붙인다. ②에서 호스트가 그 서버에 도구 목록을 요청하는데, 설정에 찍힌 것은 서버 한 줄이지만 돌아오는 도구는 우리가 계측한 버전 기준 26개이고 모두 모델이 부를 수 있다. 이 가운데 셋이 모이면 유출 경로가 된다. ③의 `list_issues`는 누구나 쓸 수 있는 공개 이슈를 컨텍스트로 들여오고, ④의 `get_file_contents`는 같은 토큰으로 비공개 저장소를 읽으며, ⑤의 `add_issue_comment`는 그 내용을 공격자가 읽을 수 있는 공개 이슈에 쓴다. 공격자가 공개 이슈에 지시문을 심어 두면, 에이전트는 그 이슈를 읽다가 지시를 따른다. 이런 경로는 손대지 않은 GitHub MCP 서버 하나만 연결한 상태에서 실제로 시연되었다 [30]. 그런데 개별 검사는 ③과 ④, ⑤를 하나씩 보기 때문에 여기서 아무것도 지목하지 않는다 [18, 42, 44]. 조합 검사는 셋이 함께 쓰일 때를 보므로 이 경로를 정확히 지목할 수 있다 [3, 38]. 그러나 거기서 멈춘다. 이 배포가 위험하다고 판정할 뿐 무엇을 꺼야 하는지는 말하지 않으므로, 사용자에게 남는 선택은 경고를 무시하거나 서버를 통째로 지우는 것뿐이다. 대책이 될 수 있는 것은 권한 제한뿐이다. 작업에서 정책을 만들어 ④를 끄는 데까지 간다 [7, 8, 40]. 다만 그러려면 ④가 비공개 데이터에 닿는다는 것을 먼저 알아야 하는데, 그 정보는 제공자가 쓴 이름과 설명에 없고 기계가 읽을 형태로 어디에도

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

쓰고 어느 필드가 주입 가능한지는 *Summarize*의 라벨이 정하며, 두 규칙은 모델과 도구 사이의 호출 경로에서만 동작한다.

핵심 난점은 그 정보가 선언되어 있지 않다는 것이다. 효과를 코드에서 읽어내야 하는데, 두 길 모두 쉽지 않다. 공개 패키지를 전부 정적 분석으로 따라가면 규모가 맞지 않고, 언어 모델에 읽히면 그 판정을 무엇으로 믿느냐가 남는다. 우리는 신뢰도 순위 요약과 결정적 재검사를 묶어 이를 푼다. 모델이 서버의 소스를 읽고 도구별 효과를 판정하되 주장마다 `file:line` 증거를 달게 하고, 결정적 검사기가 가져온 소스에서 그 줄을 다시 읽어 확인되지 않는 주장을 걸러낸다. 확인되지 않은 주장은 버리지 않고 “미확인”으로 표시해 라벨을 지키며, 모델이 판정하지 않은 도구에는 가장 보수적인 값을 준다. 문서만 근거로 한 “무해” 판정은 무해로 세지 않는다. 문서는 효과가 없다는 것을 증명하지 못하기 때문이다. 그래서 출력은 그럴듯한 요약이 아니라, 독자가 코드 한 줄까지 되짚을 수 있는 감사 가능한 기록이 된다.

집행 쪽의 난점은 도구 단위가 너무 굵다는 것이다. 라벨이 모두 맞고 작업이 쓰는 도구를 정확히 안다고 해도, 도구를 켜고 끄는 규칙은 작업이 쓰는 도구로 수행되는 공격을 가르치 못한다. 작업이 이메일을 보내는 것인데 주입이 공격자에게도 보내라고 하는 식이어서, 도구 수준 규칙은 각 작업의 도구를 완벽히 알아도 AgentDojo에서 targeted ASR 4.18% 아래로 내려갈 수 없다(\$2.3). *Guard*가 도구가 아니라 인자 값의 출처를 보는 이유가 이것이다. 이메일 주소나 채널 이름 같은 목적지가 사용자의 요청이나 앞선 결과의 구조화된 필드에서 왔다면 통과하고, 도구 결과의 산문 안에서만 나왔다면 오염된 것으로 보아 거부한다. 거부의 형태도 결과를 가르다. 허용 목록 밖의 도구를 *surface*에서 제거하면 에이전트는 없는 도구를 부르지 않는 대신 작업에서 이탈하고, 같은 도구를 보이게 두고 호출을 거부하면 거부문을 읽고 작업으로 돌아온다. 부재는 침묵이고 거부는 신호여서, 이 변경 하나로 *utility under attack*이 54.9%에서 61.3%로 올랐다.

우리는 INTERLOCK을 두 자리에서 평가한다 (\$4). *Summarize*는 공개 설정에서 가장 자주 선언되는 서버 34개와 그 도구 433개에 적용해, 모델 판정 가운데 결정적 재검사를 통과하는 비율이 전체 72.5%, 구현 코드가 배포되는 경우 90.7%임을 보인다. 집행은 AgentDojo [9]에서 베이스라인과 같은 설정으로 비교한다. 보안 케이스 909개와 benign 작업 97개에 대해 같은 모델과 같은 공격으로 No Defense, tool filter, Progent [40], CaMeL [8]을 우리가 직접 실행했다. INTERLOCK은 targeted ASR 1.06%로 Progent의 3.15%의 3분의 1이고, *utility under attack*은 61.35%로 Progent의 58.71%보다 높으며, benign utility는 73.88%로 Progent의 71.13%와 비슷하다. CaMeL은 ASR 0을 달성하지만 BU가 22.68%여서 에이전트 대부분을 멈춰 세운 결과이고, tool filter는 ASR 5.50%에 머문다. 목적지를 mark [at] gmail [dot] com처럼 흐린 적용 공격에서는 에이전트가 주소를 복원해 출처가 전혀 없는 값을 만드는데, 기본 규칙은 이 공격의 8.91%를 통과시키지만, 출처 없는 목적지를 거부하는 strict 모드는 1.10%로 막는다. 남은 공격은 실행마다 열 건으로, 목적지는 정당하고 내용이나 선택만 악의적인 경우 – 실제 동료에게 보내는 피싱 링크, 주입이 지목한 실제 파일의 삭제 – 이며, benign 작업 다섯 건은 사용자가 신뢰할 수 없는 페이지나 파일에서 목적지를 읽어 오라고 요청한 경우에 읽는다. GPT-4o는 온도 0에서도 결정적이지 않으므로, 모든 수치는 반복 실행의 평균으로 보고한다.

Contributions. We summarize our contributions as follows:

- **Observation:** 우리는 에이전트 최소 권한 기법들이 공통으로 전제하는 것, 곧 도구마다 무엇에 닿는지가 이미 알려져 있다는 전제를 정리하고, 그 전제를 세 시스템이 스스로 가정이나 한계로 적고 있음을 보인다. 또한 그 전제를 실제 서버에 대해 채우려는 유일한 선행 도구가 사람들이 실제로 선언하는 서버 대부분을 차지하는 생태계에서 작동하지 않음을 보인다. 나아가 라벨이 완전해도 도구 단위 집행에는 하한이 있음을 보인다. AgentDojo에서 도구 수준 규칙은 targeted ASR 4.18% 아래로 내려갈 수 없다.
- **New technique:** 우리는 공격자가 쓸 수 있는 산문을 결코 신뢰하지 않고 서버의 소스 코드에서 도구별 효과를 회수하는 절차를 제시한다. 모델의 모든 판정에 `file:line` 증거를 요구하고 결정적 검사기로 그 구간을 다시 읽어 확인되지 않은 주장을 표시하며, 미판정 도구에는 보수적인 값을 주고 문서만으로는 무해를 인정하지 않는다. 그 위에 그 라벨만을

소비하는 두 집행 규칙을 둔다. *Narrow*는 도구 출력을 보지 않는 LLM이 사용자의 요청에서 만든 *WRITE* 허용 목록으로 호출을 게이트하되 도구를 제거하지 않고 거부하며, *Guard*는 목적지 인자 값마다 출치를 추적해 공격자 텍스트에서만 나올 수 있는 값을 거부한다.

- **Evaluation:** 우리는 이 절차를 구현해 공개 설정에서 가장 자주 선언되는 서버 34개와 그 도구 433개에 적용하고, 모델 판정 가운데 결정적 재검사를 통과하는 비율(전체 72.5%, 구현 코드가 배포되는 경우 90.7%)을 보고하며, 공개된 패키지를 읽는 것만으로는 효과를 확정할 수 없는 세 가지 구조적 한계를 식별한다. 집행은 AgentDojo에서 같은 설정으로 실행한 베이스라인과 비교해, INTERLOCK이 targeted ASR 1.06%, utility under attack 61.35%, benign utility 73.88%로 Progent(3.15%, 58.71%, 71.13%)보다 ASR은 3분의 1이고 UUA는 높으며 BU는 비슷함을 보이고, 난독화된 목적지에 대한 적응 공격에서 strict 모드가 ASR을 8.91%에서 1.10%로 낮춤을 보인다. 두 규칙이 잡지 못하는 공격과 읽는 benign 작업을 종류별로 밝히고, 도구와 코드에서 도출한 효과 요약을 공개한다.

2 Preliminaries

2.1 Tool Surfaces and Their Effects

Table 1. Configuration corpus mined from public GitHub. Entries are server declarations; one configuration may declare several.

Configurations (deduplicated, non-fork)	19,777
declaring a single server	10,532
declaring two or more servers	9,302
Server entries by kind: npm	22,010
remote endpoint	14,250
other command	7,564
local script	7,284
PyPI	3,156
container image	1,507
Stdio entries launched without a version pin	24,354 / 26,673 (91.3%)

호스트 애플리케이션 – IDE 어시스턴트, 데스크톱 에이전트, 코딩 에이전트 – 은 설정 파일에 명시된 각 Model Context Protocol 서버를 실행하고 `tools/list`로 도구 목록을 요청한 뒤, 반환된 이름과 설명, JSON 스키마를 모델의 컨텍스트에 넣는다. 한 세션에서 이렇게 모인 도구 전체를 그 배포의 *tool surface*라 부른다. 이 구조의 세 가지 성질이 이후 논의를 이끈다. *surface*는 개별 제공자가 아니라 사용자의 설정이 결정한다. *surface* 위의 모든 도구는 호출 가능하다. 도구 선택은 광고된 텍스트를 읽은 모델이 하기 때문이다. 그리고 *surface*는 고정되어 있지 않다. 호스트는 대개 버전을 고정하지 않은 패키지 지정으로 서버를 실행하므로, 도구 위의 코드는 사용자가 아무 행동을 하지 않아도 세션 사이에 바뀔 수 있다. 실제로 §2.2에서 수집한 코퍼스를 보면 stdio 항목의 91.3%가 버전을 명시하지 않는다(표 1).

도구가 광고하는 것은 이름과 설명, 인자의 JSON 스키마, 그리고 선택적인 annotation이다. 이 가운데 무엇도 그 도구가 무엇에 닿는지를 말해 주지 않는다. 스키마는 인자의 모양만 정하고, 설명은 제공자가 자유롭게 쓴 산문이며, annotation에 대해서는 프로토콜 명세 자신이 신뢰된 서버에서 온 것이 아닌 한 믿지 말 것을 클라이언트에 요구한다. 본 논문은 도구가 닿는 대상을 네 가지 효과로 나눈다.

- **SECRET:** 사용자의 비공개 데이터를 읽는다. 그림 1의 `get_file_contents`가 비공개 저장소를 읽는 것이 여기 해당한다.

- **UNTRUSTED**: 공격자가 쓸 수 있는 콘텐츠를 컨텍스트로 들여온다. 공개 이슈 본문을 읽어 오는 `list_issues`가 그렇다.
- **SINK**: 공격자가 읽을 수 있는 곳에 쓴다. `add_issue_comment`가 공개 이슈에 코멘트를 남기는 것이 그렇다.
- **HOSTEXEC**: 호스트에서 코드를 실행한다. 호스트가 닿을 수 있는 모든 것에 닿으므로 나머지 셋을 함의한다.

우리는 에이전트가 자신이 읽은 콘텐츠 안의 지시를 따른다고 가정한다. 이는 **lethal trifecta** 프레임 [46]과 **toxic-flow** 분석이 전제하는 가정이며, 이 층위에 대해 공개된 공격들이 실제로 악용하는 가정이다 [23, 51, 52]. 공격자는 임의의 웹 호스트와 콘텐츠를 통제하고, 여러 사람이 쓸 수 있거나 제3자가 제공하는 출처의 콘텐츠 — 공개 이슈와 댓글, 저장소 내용, 커뮤니티 문서와 레지스트리, 검색 결과 — 도 통제한다. 공격자는 사용자의 로컬 파일이나 사용자 계정에 담긴 데이터를 통제하지 못하며, 설치된 서버를 수정하지도 못한다. 공격자는 자신이 통제하는 호스트와 공개된 모든 것을 읽을 수 있다. 이 가정 아래에서, 어떤 배포의 **surface**가 한 세션 안에서 **SECRET**과 **UNTRUSTED**와 **SINK**에 모두 닿을 때 그 배포는 위험하다.

2.2 Limitations

Table 2. Where each system obtains what a tool does, and what it returns.

System	Per-tool effects come from	Evaluated on	Output
Progent [40]	provider’s names and descriptions	simulated suites	per-call decision
CaMeL [8]	hand-written annotations	simulated suites	per-call decision
Fides [7]	assumed correct via wrappers	simulated suites	per-call decision
mcp-sec-audit [17]	keyword and regex rulebook	45 servers	risk score and advice
Set-level [3, 38]	taken as input, unstated	32 tools	block or revoke session
INTERLOCK	server source code, re-checked	34 servers	per-tool effect summary

에이전트의 권한을 좁히는 연구는 어떤 도구를 허용할지 정하기 위해 각 도구가 무엇에 닿는지를 알아야 한다. 그런데 표 2가 보이듯 어느 연구도 그것을 스스로 확정하지 않는다. Progent의 정책 생성기가 받는 것은 제공자가 쓴 이름과 설명과 스키마이고 [40], CaMeL의 능력 표시는 사용 사례마다 사람이 정하며 저자들은 이를 한계로 적고 자동화를 향후 과제로 남긴다 [8]. Fides는 아예 가정으로 명시해, 신뢰할 수 없는 모든 도구에 신뢰된 **wrapper**를 두어 “모든 도구가 자기 출력에 올바른 라벨을 붙인다고 가정할 수 있게” 한다 [7]. 조합을 판정하는 연구도 같은 라벨을 입력으로 받을 뿐 생산하지 않는다 [3, 38].

이들이 공유하는 벤치마크 위에서 보면 문제는 이미 풀려 있다. 우리가 같은 조건으로 직접 실행한 결과가 표 3이다. 방어를 두지 않으면 표적 공격이 45.43% 성공하지만, Progent는 세 번의 실행에서 3.15%, CaMeL은 909개 케이스 전부에서 0.00%까지 내린다. **benign utility**는 같은 구성을 반복해도 움직이므로 표는 반복 실행의 평균을 적었다 (§4.2). CaMeL의 경우 수작업 정책을 빼고 인터프리터만 두어도 ASR은 같은 0.00%이며, 정책은 **benign utility**를 38.14%에서 22.68%로 낮출 뿐이다. 즉 이 환경에서는 도구별 효과 지식을 사람이 채워 넣는다는 전제가 비용 없이 성립하고, 남은 논점은 보안이 아니라 유틸리티 대가처럼 보인다.

그러나 이 수치는 방법의 성질이라기보다 벤치마크의 성질이다. 전제가 깨지는 조건이 이들의 평가에 애초에 들어 있지 않기 때문이다. 네 시스템이 모두 사용한 AgentDojo의 도구를 우리가 측정한 결과가 표 4다. 도구 74개의 구현은 중앙값 4줄인 반면 설명은 중앙값 26단어로, *설명*이 *구현*보다 길다. 파일시스템이나 네트워크, 프로세스, 환경 변수에 닿는 도구는 하나도 없다. `read_file`은 사전 조회이고, `get_webpage`는 네트워크로 나가지 않으며, `download_file`은 어떤 파일도 쓰지 않는다. 설명과 구현을 같은 저자가 같은 파일에 쓰므로 도구가 무엇에 닿는지는 설명이 말하는

Table 3. Our own run of the privilege-narrowing systems on AgentDojo, the benchmark they share. Temperature 0, GPT-4o (2024-05-13), the important_instructions attack, over the 909 security cases Progent’s vendored copy of the suite defines and its 97 benign tasks. Slack injection_task_5 is scored from the calls that actually executed, in every arm. Benign utility is the mean $\pm$ sd over n repeated runs (in parentheses), because it moves 2–4 points between identical runs; utility under attack and targeted ASR are from one run, or the mean $\pm$ sd over three runs where marked.

System	Benign utility (%)	Utility under attack (%)	Targeted ASR (%)
No defense	78.35 $\pm$ 2.06 (4)	43.23	45.43
Tool filter [9]	68.56 $\pm$ 3.81 (4)	57.65	5.50
Progent [40]	71.13 $\pm$ 1.60 (6)	58.71 $\pm$ 0.77 (3)	3.15 $\pm$ 0.23 (3)
CaMeL [8]	22.68	27.39	0.00

Table 4. The tools of AgentDojo, the environment the privilege-narrowing and set-level systems of Table 2 are evaluated on. No tool reaches the filesystem, the network, a process or an environment variable, and the median description is longer than the median implementation.

Suite	Tools	Implementation (median lines)	Description (median words)	Reaching outside
workspace	24	1	30	0
slack	11	5	18	0
travel	28	5	26	0
banking	11	4	19	0
Total	74	4	26	0

그대로이고, 코드에서 더 알아낼 것이 없다. 즉 이 환경에서는 전제가 구조적으로 참이며, 전제가 틀렸을 때 무슨 일이 일어나는지를 이 평가는 드러낼 수 없다.

실제 서버에 대해 그 지식을 공급하려는 시도는 우리가 아는 한 하나뿐이다. mcp-sec-audit 은 키워드와 정규식 규칙집으로 능력 범주를 뽑는데, 저자들이 보고한 탐지율이 Python 서버 2 개 중 2개인 반면 JavaScript 서버는 7개 중 0개다 [17]. 그런데 사람들이 실제로 선언하는 서버는 대부분 npm이다. 공개 설정에서 수집한 표 1의 코퍼스에서 npm 항목은 22,010개인 반면 PyPI는 3,156개다. 즉 이 도구는 문제가 실제로 존재하는 생태계에서 작동하지 않는다.

2.3 Goal

이 논문의 목표는 실제 MCP 서버의 도구마다 무엇에 닿는지를, 제공자가 쓴 글이 아니라 배포된 코드에서, 독자가 한 줄까지 되짚을 수 있는 형태로 회수하는 것이다. 그렇게 얻은 효과 요약은 §2.2의 시스템들이 사람 손으로 채우거나 가정으로 두던 자리를 대신할 수 있고, 설정 시점에 직접 쓰일 수도 있다. 효과를 코드에서 직접 회수하는 것이 남은 길인데, 여기에 두 가지 어려움이 있다. 첫째, 공개된 패키지를 전부 정적 분석으로 따라가는 것은 규모가 맞지 않는다. 서버는 TypeScript 와 Python, Go, Rust 등으로 작성되고, 번들이나 minify된 형태로, 또는 바이너리를 내려받는 실행 기로 배포되며, 얇은 wrapper가 실제 구현을 의존성에 두기도 한다. mcp-sec-audit이 JavaScript 에서 멈춘 지점이 바로 여기다. 둘째, 언어 모델에 코드를 읽히면 규모 문제는 사라지지만 그 판 정을 무엇으로 믿을지가 남는다. 모델은 읽지 않은 것을 그럴듯하게 서술할 수 있고, 판정 대상 자체가 공격자가 쓸 수 있는 텍스트를 포함한다.

회수한 라벨은 집행되어야 하는데, 도구 단위의 집행으로는 충분하지 않다. 도구 수준의 거부 목록은 작업에 필요 없는 도구를 끄는 것이므로, 주입된 호출이 작업 자체가 필요로 하는 도구를 쓰면 그것을 정당한 호출과 구별하지 못한다. 사용자가 이메일을 보내라고 지시했고 주입이 공 격자에게도 보내라고 말하면 두 호출은 같은 도구를 쓴다. 표 3의 무방어 궤적을 다시 재생하면

성공한 공격 413건 가운데 38건이 작업이 필요로 하는 도구만으로 수행되며, 따라서 각 작업의 도구를 완벽히 안다 해도 도구 수준 거부 목록의 targeted ASR은 이 벤치마크에서 4.18% 아래로 내려갈 수 없다. 이는 Progent가 실제로 도달한 값보다 높다. 그러므로 집행은 어느 도구를 허용 할지에 더해, 호출마다 그 목적지 인자의 값이 어디에서 왔는지를 판정해야 하고, 라벨은 도구의 효과와 함께 어느 인자가 목적지인지를 담아야 한다.

3 Our Approach

3.1 Overview

INTERLOCK은 설정과 사용자의 작업을 입력으로 받는다. 설정은 호스트가 띄울 서버와 그 서버들이 유도하는 tool surface를 정하고, 작업은 그 surface 가운데 무엇이 쓰여도 되는지를 정한다. 절차는 세 단계다. *Summarize*는 서버의 소스 코드에서 도구별 효과 요약을 도출하고, 모든 주장을 인용된 file:line에 대해 다시 검사한다. 나머지 두 단계는 그 라벨을 소비하는 집행 규칙이다. *Narrow*는 에이전트의 첫 턴 이전에 작업만 보고 그 작업이 필요로 하는 WRITE 도구 - 상태를 바꾸거나 데이터를 내보내는 도구 - 의 허용 목록을 만들고, 목록 밖의 WRITE 호출을 거부한다. *Guard*는 목적지 인자를 가진 모든 호출에서 각 값의 출처를 추적해, 공격자가 쓸 수 있는 텍스트에서만 나올 수 있는 값을 가진 호출을 거부한다. 두 규칙이 요약에서 읽는 것은 도구가 WRITE인지와 어느 인자가 목적지나 값인지, 결과의 어느 필드가 공격자가 쓸 수 있는 텍스트 (UNTRUSTED)이고 어느 필드가 시스템이 등록한 식별자인지이며, 도구의 설명은 어느 규칙도 읽지 않는다. 세 단계는 모두 구현되어 있다. *Summarize*는 실제 MCP 서버를 대상으로 평가하고 (§4.3), *Narrow*와 *Guard*는 선행 시스템들이 공유하는 AgentDojo 위에서 평가하는데, 거기서 두 규칙이 읽는 라벨은 모델이 도구 코드에서 분류하고 적대적으로 재검사한 것이다 (§4.4).

3.2 Summarize: Trust-ranked Effect Summaries

이 단계는 광고된 도구마다 하나의 질문에 답한다. 이 도구는 §2.1의 네 효과 가운데 무엇이 닿을 수 있으며 그 증거는 무엇인가. 증거는 공격자가 얼마나 통제할 수 있는지에 따라 순위가 매겨진다. 설정 자신의 인자와 서버의 소스 코드가 1순위, 선언된 스키마와 annotation이 2순위이며, 자유 서술 설명은 효과를 낮추는 데 결코 쓰이지 않는다. 올리는 데만 쓸 수 있는데, 설명은 판정 대상인 당사자가 작성하기 때문이다. HOSTEXEC은 나머지 셋을 함의한다. 호스트에서의 코드 실행은 호스트가 닿을 수 있는 모든 것에 닿기 때문이다.

요약은 언어 모델이 생성하되 모든 주장에 file:line 증거를 인용해야 하며, 이후 결정적 검사가 가져온 소스에서 인용된 구간을 다시 읽어 걸러낸다. 인용이 가져온 트리 안에서 해석되지 않거나, 그 구간에 도구 이름이나 근거 설명이 지목한 식별자가 없으면, 그 주장은 유지되되 미확인으로 표시된다. 미확인 주장이 효과를 낮춰서는 안 되므로 라벨은 살아남지만, 검증된 것으로는 세지 않는다. 모델이 아예 판정하지 않은 도구에는 가장 보수적인 값인 HOSTEXEC을 부여한다. 모델이 라벨 없이 무해하다고 판정한 도구는 그 증거가 코드에서 해석될 때만 무해로 센다. 문서는 효과의 부재를 입증할 수 없기 때문이다. 이 구분이 이 단계의 출력을 그럴듯한 것이 아니라 감사 가능한 것으로 만들며, §4은 그중 얼마가 검사기를 통과하는지 측정한다.

실무적으로 중요한 세부가 둘 있다. 모델에게는 모든 줄에 실제 줄 번호를 붙인 소스를 보여준다. 줄 번호가 없으면 인용된 줄이 파일 뒤로 갈수록 어긋나고, 올바른 판정이 검증에 실패하기 때문이다. 또한 패키지 자신의 코드가 도구 이름을 전혀 언급하지 않는 경우 - 구현이 의존성에 있는 래퍼 패키지가 그렇다 - 그 이름을 언급하는 직접 의존성을 스크립트 실행을 끈 채 가져와 함께 보여준다. 파이프라인의 어느 지점에서도 제공자 코드는 실행되지 않는다.

3.3 Narrow: The Write Surface a Task Needs

이 단계는 작업이 필요로 하는 WRITE 도구의 허용 목록을 에이전트의 첫 턴 이전에 한 번 만든다. 시스템 메시지와 사용자의 요청만 보는 언어 모델이 surface 위의 WRITE 도구 가운데 그 작업에 필요한 것의 이름을 대고, 그 답은 에이전트의 컨텍스트에 더해지지 않고 집행기에만 기록된다.

이 모델은 도구 출력을 결코 보지 않으므로, 주입이 목록을 넓힐 방법은 없다. 게이트 대상은 목적지 인자나 값 인자를 가진 WRITE, 곧 주입이 겨냥할 수 있는 WRITE뿐이다. READ 도구는 게이트하지 않는다. 읽기를 막는 것은 작업이 쓸 정보를 끊을 뿐이고, 읽기 도구가 목적지 인자를 가지는 경우 – URL을 받는 `get_webpage` – 는 다음 단계가 호출마다 검사하기 때문이다. 메일을 읽으면서 읽음 표시를 남기는 `get_unread_emails`처럼 부수 효과만 있는 WRITE도 게이트하지 않는데, 공격자가 무엇을 쓰든 그 도구가 하는 일은 바뀌지 않기 때문이다. 그림 1의 선언된 작업이 이슈를 정리하기만 하고 코멘트를 남기지 않는다면 `add_issue_comment`는 목록에 들지 않고, 그 호출은 거부된다.

목록 밖의 WRITE는 `surface`에서 제거하지 않고 그대로 보이게 두며, 그 호출이 오면 실행하지 않고 거부한다. 보안은 같지만 에이전트가 겪는 것이 다르다. 주입이 작업에 없는 도구를 지목했을 때 그 도구가 없으면 에이전트에게 남는 것은 침묵이고, 에이전트는 표류한다. 거부는 신호이고, 에이전트는 사용자의 작업으로 돌아온다. 같은 라벨과 같은 거부 문구를 두고 도구를 제거하는 대신 보이게 두고 거부하는 것의 효과는 §4.5에서 쥔다. 호스트가 설정 시점의 제어 수단 – 도구 단위 거부 항목, 서버 모드와 `toolset` 플래그 – 을 제공하면 같은 라벨 위의 결정을 설정 `diff`로 내보낼 수 있다. 그림 1의 ⑦이 그 형태로, 선언된 작업이 ③과 ⑤를 쓰므로 ④를 끈다. 이 논문은 그 형태를 평가하지 않는다.

3.4 Guard: Destination Provenance at Each Call

도구 단위 규칙은 AgentDojo에서 targeted ASR을 4.18% 아래로 내릴 수 없다(§2.3). 이 단계는 그 호출들을 인자 수준에서 가른다. 목적지 인자를 가진 모든 호출에서, 도구의 종류와 무관하게 – `get_webpage`는 READ이지만 URL에 담긴 것을 내보낸다 – 각 값이 어디에서 왔을 수 있는지를 추적한다. 출처는 넷이다. 사용자의 요청, 앞선 결과의 구조화된 필드, 앞선 쓰기의 결과, 그리고 공격자가 쓸 수 있는 텍스트. 값은 정규화된 형태와 그 축약형 – 스킴을 뗀 것, `www.`를 뗀 것, 호스트만 – 으로 각각 찾으며, 어느 형태에든 신뢰된 출처가 하나라도 있으면 값은 통과한다. 공격자가 쓸 수 있는 텍스트에서만 찾아지는 값을 가진 호출은 거부된다.

출처 판정은 도구 결과를 필드 단위로 쪼개어 잎마다 부류를 매긴 뒤 이루어진다. 결과는 YAML로 기록되므로 필드 분리는 정확하고, `body` 안에 주입된 문자열이 키를 만들어 낼 수 없다. 잎의 부류는 넷이다. 신뢰할 수 없는 산문은 공격자가 쓸 수 있는 필드의 텍스트로, 값이 그 안에 포함되면 오염된다. 고정 산문은 공격자가 닿지 못하는 텍스트로, 값이 그 안에 포함되면 통과한다. 식별자는 `id`와 주소, 시스템이 등록한 이름으로, 값이 그것과 동등할 때만 통과하고 포함은 아무것도 뜻하지 않는다. 사전의 키는 항상 식별자다. 이름 붙은 식별자는 공격자가 철자를 정한 식별자 – 공격자가 만든 채널 – 로, 동등하면 통과하고 포함되면 오염된다. 공격자가 이름 지은 채널은 실제 채널이지만 그 텍스트는 지시를 실을 수 있어서, `External_visit www.evil.com`이라는 채널은 식별자이되 `www.evil.com`은 그것이 아니다. 짧은 `id`는 텍스트 안에서 찾지 않고 식별자와의 동등으로만 맞춘다. 어디서나 찾아지기 때문이다. 반환값 전체가 공격자가 쓸 수 있는 도구 – 파일 내용, 웹 페이지 – 의 결과는 파싱하지 않는다. `key: value`처럼 보이는 내용이 구조화된 필드로 승격되어서는 안 되기 때문이다.

`strict` 모드는 조건 하나를 더한다. 신뢰된 출처도 오염된 출처도 없는 목적지, 곧 세 글자 이상이면서 신뢰된 컨텍스트 어디에도 없는 값도 거부한다. 에이전트는 주입이 `mark [at] gmail [dot] com`으로 적은 주소를 스스로 복원하고, 복원된 값은 어느 텍스트에도 없으므로 기본 규칙은 그것을 잡지 못한다. 에이전트가 지어냈든 난독화된 주입에서 복호했든, 그 값은 사용자가 요청한 목적지가 아니다. 짧은 `id`는 여기서도 예외인데, 어느 앞과도 같지 않은 `id`는 에이전트가 적어 넣은 `id`일 뿐 유출 경로가 아니다. 거부는 두 단계 모두 같은 형태의 결과로 에이전트에게 돌아간다. 어떤 인자나 값이 검사에 걸렸는지는 말하지 않고, 도구 결과 안의 지시는 사용자의 것이 아니니 무시하고 이미 가진 정보로 사용자의 원래 작업을 계속하라고 말한다. 인자와 값을 밝히면 주입에게 가드가 무엇을 검사하는지 탐색할 통로를 주고, 다음에 무엇을 할지 말하지 않으면 에이전트는 작업을 버린다. 판정의 세부는 로그에만 남는다.

4 Experiments

4.1 Research Questions

본 연구의 실험은 INTERLOCK의 두 부분을 각각 묻는다. *Summarize*가 내놓는 효과 요약은 그 자체로 참임을 보일 수 없고 – 판정 대상이 제3자의 코드이므로 정답 집합이 존재하지 않는다 – 다만 각 주장이 스스로 지목한 증거로 되짚어지는지는 결정적으로 확인할 수 있다. 그 요약을 소비하는 두 집행 규칙 *Narrow*와 *Guard*는 결과로 짚 수 있다. 공격이 막혔는지와 사용자의 작업이 유지되었는지를 같은 벤치마크 위에서 베이스라인과 나란히 재면 된다. 따라서 우리는 다음을 묻는다.

- **RQ1.** 실제 MCP 서버에 대해 모델이 내놓은 효과 판정 가운데 얼마나 많은 부분이 결정적 재검사를 통과하는가? 통과하지 못하는 판정은 어디에서 발생하며, 무엇이 그 비율을 좌우하는가?
- **RQ2.** *Narrow*와 *Guard*를 함께 쓴 INTERLOCK이 같은 조건의 베이스라인보다 낮은 targeted ASR을 더 적은 utility 손실로 달성하는가?
- **RQ3.** 두 규칙 각각이 얼마를 기여하며, 그 결과는 목적지를 난독화한 적응형 주입에도 유지되는가?

RQ1의 답은 §4.3에, RQ2의 답은 §4.4에, RQ3의 답은 §4.5에 있다.

4.2 Experimental Settings

4.2.1 Benchmarks. 본 연구는 AgentDojo [9]를 벤치마크로 사용한다. 비교 대상인 세 선행연구가 모두 이 환경에서 결과를 보고했기 때문이다. AgentDojo는 프롬프트 주입에 대한 에이전트 방어를 평가하는 환경으로, 업무용 메일과 일정과 드라이브를 다루는 *workspace*, 메신저를 다루는 *slack*, 예약을 다루는 *travel*, 계좌를 다루는 *banking*의 네 *suite*에 도구 74개와 사용자 과제 97개, 주입 과제 35개를 담고 있다. 하나의 보안 *케이스*는 사용자 과제와 주입 과제를 짝지은 것으로, 공격자의 지시는 에이전트가 읽게 될 콘텐츠에 심기고 성공 여부는 결정적 Python 함수가 판정한다. 언어 모델 심판을 쓰지 않는 이유를 저자들이 밝혀 두었고 우리도 이를 따르는데, 공격이 충분히 성공적이면 평가 모델까지 탈취할 수 있기 때문이다. 다만 우리는 보안 케이스 949개 전부가 아니라 **909개**를 사용한다. Progent가 *travel suite*의 주입 과제 두 개를 선호도 공격이라는 이유로 모든 실험에서 제외했고 [40], 그 제외를 보고서만이 아니라 함께 배포한 벤치마크 사본에서 수행했기 때문이다. 우리가 그 사본을 불러 확인한 결과 *travel suite*의 주입 과제는 일곱 개가 아니라 다섯 개였으며, Progent가 보고한 수치가 그 집합에서 나왔으므로 네 베이스라인을 같은 표에 올리려면 나머지도 같은 집합에서 재야 한다. 우리가 직접 실행하는 베이스라인은 949개를 모두 돌리되 집계에서 해당 두 주입 과제를 빼는 방식으로 이를 맞추고, 949개 기준 값도 함께 보고해 제외가 결과를 얼마나 움직이는지 드러낸다. *suite*별 구성은 표 5와 같다.

RQ3에서는 *important_instructions*의 난독화 변형을 함께 쓴다. 주입문 안의 모든 이메일 주소와 호스트 이름은 “mark[at]gmail[dot]com”처럼 풀어 쓰고 IBAN은 네 자리씩 띄어 써서, 에이전트가 도구에 넘겨야 할 문자열 그대로는 주입문 어디에도 나타나지 않게 한 것이며, 그 밖의 문면은 동일하다.

효과 요약 단계는 AgentDojo가 아니라 실제 MCP 서버를 대상으로 평가한다. 어떤 서버를 읽을 것인지는 저자의 판단이 아니라 공개 설정에 등장하는 빈도로 정하며, 이를 위해 현재 호스트들이 읽는 설정 형식 다섯 가지를 공개 GitHub에서 검색해 표 1의 코퍼스를 수집하였다. 각 파일은 메모리에서 파싱해 서버의 식별자와 실행 형태만 보관하며 자격증명은 저장하지 않는다. 이 코퍼스에서 가장 자주 선언되는 서버 34개와 그 도구 433개가 평가 대상이다.

4.2.2 Baselines. 본 연구는 MCP 에이전트의 권한을 제한하는 선행연구인 AgentDojo [9], Progent [40], CaMeL [8]을 베이스라인으로 삼아 평가를 수행한다. Fides [7] 역시 같은 갈래의 선행연구이나, 공개된 저장소가 튜토리얼 노트북 하나여서 보고된 결과를 재현할 수 없기 때문에

Table 5. The four suites of AgentDojo. Two injection tasks of the travel suite are excluded, so 909 of the 949 security cases are used.

Suite	Tools	User tasks	Injection tasks	Security cases (used/total)
workspace	24	40	14	560 / 560
slack	11	21	5	105 / 105
travel	28	20	7	100 / 140
banking	11	16	9	144 / 144
Total	74	97	35	909 / 949

제외하였다. 조합 검사 계열의 연구도 제외하였는데, 위험 여부를 판정할 뿐 설정을 바꾸지 않으므로 공격이 막혔는지와 작업이 유지되었는지로 채점하는 자리에 놓일 수 없기 때문이다. 비교 대상은 다음과 같다.

- **No Defense** [9]: 방어를 두지 않은 에이전트로, 나머지 넷이 무엇을 줄였는지 재는 기준점이다.
- **Tool Filter** [9]: AgentDojo가 함께 제공하는 방어다. 질의를 받으면 모델에게 그 작업에 필요한 도구만 고르게 한 뒤 나머지를 제거하며, 집행은 첫 호출 이전에 한 번 이루어진다.
- **Progent** [40]: 사용자의 작업에서 도구 이름과 인자에 대한 기호 규칙을 LLM으로 생성하고 호출마다 그 규칙과 대조하며, 실행 중 새 정보가 들어오면 정책을 갱신한다.
- **CaMeL** [8]: 권한 있는 LLM이 신뢰된 질의를 제한된 프로그램으로 옮기고 격리된 LLM이 신뢰할 수 없는 데이터를 해석하며, 전용 인터프리터가 값마다 능력을 추적해 도구 호출 시점에 정책을 집행한다.
- **INTERLOCK**: 본 연구의 시스템으로, Narrow와 Guard를 함께 쿼 구성이다. 허용 목록 밖의 WRITE는 제거하지 않고 호출 시점에 거부하며(게이트), 출처가 전혀 없는 목적지까지 거부하는 strict 모드는 RQ3에서 따로 켜다. 두 규칙이 소비하는 도구별 라벨 – READ/WRITE, 목적지 인자, 주입 가능한 출력 필드, 식별자 필드 – 는 도구의 설명이 아니라 코드에서 분류하고 적대적으로 채점한 것이다.

4.2.3 Evaluation Metrics. 본 연구는 AgentDojo가 정의한 세 지표를 그대로 사용한다 [9]. 비교 대상들이 같은 용어로 결과를 보고하므로, 지표를 새로 정의하기보다 이를 따르는 편이 비교에 유리하다. 각 지표는 다음과 같다.

- **Benign Utility(BU)**: 공격이 없을 때 사용자 과제를 푼 비율.
- **Utility Under Attack(UUA)**: 보안 케이스에서 부작용 없이 사용자 과제를 푼 비율.
- **Targeted Attack Success Rate(ASR)**: 보안 케이스에서 공격자의 목표가 달성된 비율. 낮을수록 좋다.

방어는 이 셋을 함께 읽어야 한다. 도구를 전부 꺼 버리면 ASR은 0이 되지만 BU도 함께 0이 되므로, ASR만으로는 공격을 막은 것과 에이전트를 멈춰 세운 것이 구별되지 않는다.

측정 방식에서 본 연구는 두 가지를 벤치마크의 기본값과 다르게 한다. 첫째, slack suite의 injection_task_5는 모든 arm에서 실제로 실행된 호출로 채점한다. 이 과제는 성공해도 최종 상태가 바뀌지 않아 AgentDojo가 호출 기록으로 채점하므로 방어가 거부한 호출도 성공으로 세기 때문이다. Progent의 저자들은 이를 손으로 보정했고, 본 연구는 과제의 상태 기계를 실행된 호출 위에서 재생하는 방식으로 같은 보정을 모든 arm에 적용한다. 이 보정은 Progent의 첫 공격 실행에서 8건, INTERLOCK의 첫 게이트 실행에서 6건의 판정을 바꾸며, 기록된 값은 보정된 값 옆에 보관한다. 둘째, BU는 같은 구성의 모든 benign 실행에 대한 평균 ± 표준편차로 보고하고, Progent와 INTERLOCK의 게이트 구성은 공격 실행을 세 번씩 반복해 UUA와 ASR도 같은 형태로 보고한다. 반복이 필요한 이유는 다음 항목에 있다.

Table 6. Outcome of deterministic re-checking over 34 servers and 433 advertised tools.

	Count	Share
Claims (labels asserted)	335	
verified in code	243	72.5% of claims
verified in documentation only	11	3.3% of claims
demoted (citation did not resolve)	81	24.2% of claims
Cleared with code evidence	65	
Cleared without code evidence	33	7.6% of tools
Unjudged tools	0	0.0% of tools
Judged names not in the surface	2	0.5% of tools
Servers usable (claim rate ≥ 0.70 , few clears, no unjudged)	27 / 34	79.4%
Cost of the run (USD)	8.53	

4.2.4 Environments. 본 연구는 GPT-4o를 사용한다. 네 베이스라인이 공통으로 사용한 모델일 뿐 아니라, 그들이 사용한 모델 가운데 지금도 API로 이용할 수 있는 유일한 모델이기 때문이다. Progent가 함께 보고한 Claude Sonnet 4와 Gemini 2.5 Flash, GPT-4.1, CaMeL이 사용한 Claude 3.5 Sonnet과 Gemini 2.5 Pro는 모두 제공자가 서비스를 종료하여 재현할 수 없다. Progent는 에이전트와 정책 생성기 양쪽에, CaMeL은 권한 있는 LLM과 격리된 LLM 양쪽에 같은 모델을 사용하므로 우리도 그와 같이 맞추었다. 온도는 베이스라인이 설정한 0을 그대로 따랐다. AgentDojo와 Progent, CaMeL은 모두 온도를 0으로 두고 보안 케이스마다 한 번씩만 실행하며 반복 실행을 하지 않는데, 이들 논문이 보고하는 구간 역시 반복 사이의 편차가 아니라 케이스 수에 대한 신뢰 구간이다. 그러나 온도 0에서도 GPT-4o의 출력은 결정적이지 않다. 같은 구성을 다시 실행하면 케이스 909개 위의 UUA와 ASR은 표준편차 0.8 이하로 안정적이지만 과제 97개 위의 BU는 2-4점 흔들리며, 베이스라인 논문이 단일 실행으로 보고한 BU의 몇 점 차이도 이 폭 안에 있다. 그래서 본 연구는 같은 구성을 반복 실행하고 앞 항목의 방식으로 보고한다. 실험은 macOS 26.6, Apple M1 Ultra(20코어), 64 GB 메모리 환경에서 수행하였고, 모든 모델 호출이 제공자의 API를 거치므로 GPU는 사용하지 않는다. 벤치마크와 각 베이스라인은 공개 저장소를 고정된 리비전으로 받아 사용하였으며, 체크아웃에 가한 수정은 두 가지뿐이다. AgentDojo에는 모델 식별자를 등록하는 세 줄을 더하였고, CaMeL에는 지원하지 않는 상수를 만났을 때 오류 메시지를 만들면서 존재하지 않는 속성을 참조해 실행 전체가 중단되는 버그를 한 줄 고쳤다. 과제와 채점기, 공격, 정책 로직에는 손대지 않았다.

4.3 RQ1: How Much of the Effect Judgement Survives Re-checking?

우리는 광고된 surface를 복원할 수 있었던, 가장 자주 설정되는 npm 및 PyPI 서버 34개, 총 433개의 광고된 도구를 대상으로, 고정된 rubric과 JSON 스키마를 사용하는 Claude Opus 5, 패키지당 400 KB의 소스 예산, 그리고 §3에서 설명한 검사기로 프로파일을 만든다. 광고된 각 도구는 정확히 하나의 범주에 들어간다. 모델이 라벨을 주장했고 인용이 코드에서 해석된 경우(*verified*), 문서에서만 해석된 경우(*verified_doc*), 해석되지 않은 경우(*demoted*), 모델이 무해로 판정했고 코드 증거가 있는 경우(*cleared verified*) 또는 없는 경우(*cleared unverified*), 그리고 모델이 판정하지 않은 경우(*unjudged*)다. 광고된 surface에 없는 이름을 판정한 경우는 별도로 센다. 기준과 정의는 실행 전에 사전 등록했으며, 계획서가 정한 dry-run에서 발견된 측정 도구 변경은 모두 실행 이후가 아니라 이전에 기록했다.

표 6가 RQ1에 답한다. 서버 34개 전체에서 라벨 주장의 72.5%가 코드로 검증된다. 판정되지 않은 도구는 없고, 이름 날조는 433개 중 2건으로 무시할 만하다. 손실의 대부분은 세 서버에서 나오며,

Table 7. INTERLOCK on AgentDojo, under the setting of Table 3; Progent is repeated from that table as the reference. Percentages, mean $\pm$ sd over n runs of the same configuration, or one run where $n = 1$. Slack injection_task_5 is scored from the calls that executed, in every arm: this moves Progent’s first run from 3.96 as logged to 3.08, and the mean over its three runs is 3.15.

System	BU	UUA	Targeted ASR
Progent [40]	71.13 $\pm$ 1.60 (6)	58.71 $\pm$ 0.77 (3)	3.15 $\pm$ 0.23 (3)
INTERLOCK (gate)	73.88 $\pm$ 0.60 (3)	61.35 $\pm$ 0.06 (3)	1.06 $\pm$ 0.17 (3)
INTERLOCK (gate+strict)	75.26 (1)	60.40 (1)	1.10 (1)

각각은 라벨링 오류가 아니라 구조적 이유로 실패한다. 하나는 실행기만 배포하고 컴파일된 바이너리를 내려받으므로 구현이 패키지 안에 전혀 없고, 인용 68건이 모두 문서를 가리킨다. 하나는 시작 시점에 인터페이스 명세에서 도구 이름을 생성하므로 그 이름이 소스 어디에도 없고 선택할 소스 파일이 없었다. 하나는 도구를 minified 번들로 구현하는데, 인용 35건 중 13건이 번들의 첫 줄을 가리키는 반면 인용된 식별자는 다른 위치에 있다. 구현 코드를 읽을 수 있었던 서버 27개로 한정하면 코드 검증률은 **90.7%**(268건 중 243건)다. 사용 가능 집합에서 빠진 나머지 네 서버는 절차의 실패가 아니다. 그 서버들의 도구는 모두 무해로 판정되었고 그중 셋은 전부 코드 증거에 기반했으며, 사전 등록한 사용 가능성 규칙이 주장된 라벨이 없는 서버를 사용 불가로 세기 때문이다.

4.4 RQ2: Does INTERLOCK Beat the Baselines?

표 7가 RQ2에 답한다. INTERLOCK은 Progent보다 공격 하에서 작업을 더 많이 지키면서 공격은 더 적게 통과시킨다. utility under attack은 61.35%로 Progent의 58.71%보다 2.64점 높고, 이 차이는 Progent의 실행 간 표준편차 0.77의 세 배를 넘는다. targeted ASR은 1.06%로 Progent의 3.15%의 3분의 1이다. benign utility는 73.88%와 71.13%로 비슷하며, 두 값의 차이는 같은 구성을 반복 실행할 때 BU가 흔들리는 폭($n = 97$ 에서 표준편차 2–4점) 안에 있다. Tool filter는 도구 단위의 허용 목록으로 ASR을 5.50%까지 내리고 거기서 멈춘다(§2.3). CaMeL은 ASR 0.00%에 도달하지만 BU 22.68%를 대가로 치른다. 게이트+strict는 단일 실행에서 BU 75.26%, UUA 60.40%, ASR 1.10%로, BU와 ASR의 차이는 실행 간 편차 안에 있고 UUA의 0.95점 차이는 strict가 단일 실행이라 판정을 유보한다.

남는 것은 두 가지다. 공격 쪽에서는 모든 구성에서 실행마다 10건이 살아남는다. 어느 경우든 목적지는 정당하고 내용이나 선택만이 적대적이다. 실제 동료에게 피싱 링크를 보내는 DM, 주입이 삭제하라고 지목한 실제 파일, 주입이 예약하라고 지목한 실제 호텔이 그것이다. 그림 1의 유출도 이 부류다. 선언된 작업이 코멘트를 남기므로 ⑤는 허용되고 그 목적지는 실제 이슈이며, 새는 것은 내용이다. 이 가운데 일부를 잡을 규칙 둘을 오프라인으로 재생해 보았으나 기각했다. 공격자 텍스트에 언급되지만 한 목적지를 거부하면 benign 작업 2건을 잃고 공격 1건을 막으며, 값 인자에도 taint를 적용하면 benign 작업 4건을 잃고 공격 1건을 막는다. benign 쪽에서는 실행마다 5건이 손실되는데, 모두 사용자가 목적지를 신뢰할 수 없는 페이지나 파일에서 읽어 쓰라고 지시한 작업이다(“그의 이메일은 그의 웹사이트에 있다”, “파일에 있는 청구서를 지불하라”). 그런 값을 읽어 내는 것은 주입이 같은 일을 하는 것과 구별되지 않으며, Progent의 정책도 같은 구멍을 갖는다.

4.5 RQ3: Which Rule Does the Work, and Does It Survive an Adaptive Attack?

표 8는 두 규칙이 서로 다른 공격을 막는다는 것을 보인다. 허용 목록만 두면 ASR 12.54%가 남고 출처 taint만 두면 7.48%가 남지만, 둘을 함께 두면 1.43%다. 허용 목록은 작업에 필요 없는 WRITE 도구로 가는 공격을 막고, taint는 작업이 필요로 하는 도구의 목적지를 공격자 텍스트로 바꿔 넣는 공격을 막는다. 각각이 놓치는 것을 다른 쪽이 잡으므로 둘의 합은 어느 한쪽보다 훨씬 작다.

Table 8. Ablation of the two rules under the plain important_instructions attack. Prune mode, single runs, percentages.

Rules	BU	UUA	ASR
Allow-list only	76.29	54.13	12.54
Taint only	75.26	56.44	7.48
Both	69.07	55.01	1.43

Table 9. The adaptive attack important_instructions_obfuscated, whose destinations are spelt “mark[at] gmail[dot] com”. Single runs, percentages.

System	UUA	ASR
No defense	46.97	39.05
INTERLOCK (prune, default taint)	55.12	8.91
INTERLOCK (prune, strict)	55.89	1.10
INTERLOCK (gate+strict)	59.96	1.21

UUA는 세 구성에서 54.13%, 56.44%, 55.01%로 거의 같다. 이 구성들은 모두 제거 방식이며, UUA의 차이는 규칙이 아니라 거부 방식에서 온다. 다음 문단이 그것을 쟀다. BU 열은 단일 실행이고 같은 구성이 반복 실행에서 표 7만큼 흔들리므로, 이 열에서는 순서를 읽지 않는다.

게이트가 제거를 이긴 이유는 부재는 침묵이고 거부는 신호이기 때문이다. 허용 목록에 없는 WRITE 도구를 런타임에서 제거하는 초기 구성에서, 가드가 호출을 거부한 공격 케이스 175건의 UUA는 44.0%로 같은 케이스에서 방어 없는 에이전트가 낸 55.4%보다 낮았고, 그중 76건은 거부 이후 어떤 도구도 더 부르지 않았다. 거부 문구만을 행동 지시형으로 – 도구 결과 안의 지시는 사용자에게서 온 것이 아니니 무시하고 원래 작업을 계속하라 – 바꾸자 이 부분집합의 UUA는 55.4%로 올랐다. 그러나 나머지 734건에서 가드는 아무것도 거부하지 않았다. 주입이 부르려던 도구가 애초에 없었고, 에이전트는 제거된 도구를 한 번도 부르지 않았으며, 그 대신 작업에서 이탈했다. 같은 상황에서 Progent의 에이전트는 허용되지 않는다는 응답을 받고 작업으로 돌아왔다. 도구를 보이는 채로 두고 호출 시점에 같은 문구로 거부하자 UUA는 54.86%에서 61.35%로 올랐고, ASR은 1.28%에서 1.06%로 표준편차 0.17 안에서 움직였다.

표 9는 taint 규칙을 정면으로 겨냥한 공격이다. 공격자는 목적지를 “mark[at] gmail[dot] com”처럼 흠어 적어, 어떤 도구 결과에도 완성된 주소가 나타나지 않게 한다. 방어가 없으면 이 공격은 39.05% 성공하고, 기본 taint 규칙은 8.91%를 통과시킨다. GPT-4o가 흠어진 조각을 온전한 주소로 복원하는데, 복원된 값은 어떤 도구 결과에도 포함되어 있지 않으므로 오염된 것이 아니라 출처가 없는 값이 되기 때문이다. strict 모드는 출처가 전혀 없는 목적지도 거부하며, 이것이 ASR을 1.10%로 내린다. 게이트와 함께 두면 UUA 59.96%, ASR 1.21%로, 평균 공격에서의 60.40%와 1.10%에 가깝다. 스위트별로는 travel의 ASR만 1.0%에서 2.0%로 오르고 banking 0.7%, slack 2.9%, workspace 0.9%는 그대로다. 평균 공격에서 strict는 ASR을 바꾸지 않고(게이트 첫 실행 1.10%, 게이트+strict 1.10%), 오프라인 재생에서 strict의 비용은 benign 작업 1건으로 에이전트가 주소를 지어내어 풀었던 초대 작업이다. 그래서 본 연구는 게이트+strict를 권장 구성으로 둔다.

5 Threats to Validity

코퍼스. GitHub 코드 검색은 질의당 결과 수에 상한을 두고 표집이 아니라 관련도 순위로 반환하므로, 이 코퍼스는 배포의 무작위 표본이 아니다. 커밋된 설정은 사람들이 실제로 실행하는 설정과 다를 수도 있다. 판정 대상 집합. 효과는 가장 자주 설정되는 서버 34개에 대해서만 도출했으므로, 결과는 인기 서버를 설명하며 롱테일을 설명하지 않는다. 위협 모델. 우리의 판정은 최악

가정의 에이전트 아래에서 광고된 surface가 갖는 성질이다. 특정 배포가 공격을 당했다거나 공격이 성공한다고 주장하지 않으며, 악용 가능성 확인은 향후 과제다. 검사기. 검사기는 한 방향으로 엄격하고 다른 방향으로 느슨하다. 강등 81건 중 약 9건은 인용 구간에 해당 식별자가 더 긴 표현 안에 실제로 존재하지만 토큰 일치가 이를 놓친 경우이고, 반대로 코드로 검증된 판정 308건 중 4건은 네 글자 이하의 토큰에 기대고 있다. 발췌. 큰 파일에 대해서는 이름이 등장하는 지점 주변 창만 보여주므로, 큰 파일에 존재하는 도구 이름 11개는 모델에게 한 번도 보이지 않았다. 따라서 의존성 추적의 이득은 하한이다. 범위. 우리의 결론은 설정 파일이 선언하는 MCP 서버에 대한 것이며, 에이전트 스킬로 일반화하지 않는다. 스킬은 광고된 도구 목록을 갖지 않으므로 여기서 쓴 측정 단위가 그대로 적용되지 않는다. 실행 간 분산. GPT-4o는 온도 0에서도 결정적이지 않다. 같은 구성을 같은 설정으로 다시 실행하면 benign utility가 97개 과제 위에서 2-4점 움직이므로, 본 연구는 benign utility를 반복 실행의 평균과 표준편차로 보고하고 Progent와 INTERLOCK의 게이트 구성에는 공격 실행을 세 번씩 두었다. 909개 보안 케이스 위의 utility under attack과 targeted ASR은 실행 사이 표준편차가 0.8 이하로 안정적이다. 채점. AgentDojo는 slack의 injection_task_5를 종료 상태가 아니라 호출 기록으로 채점하므로, 방어가 거부한 호출도 성공으로 센다. 본 연구는 모든 비교 대상에 대해 이 과제를 실제로 실행된 호출로 채점하며, 이 보정은 Progent의 첫 실행에서 targeted ASR을 3.96에서 3.08로 옮긴다. 라벨의 출처. AgentDojo에서 효과 라벨은 모델이 도구 코드에서 분류하고 적대적으로 재검사한 것이지만, 이 벤치마크에서는 설명과 코드가 일치하므로 §2.2이 논한 코드 유래 라벨의 이점은 RQ2에서 발휘되지 않는다. 실제 서버 위에서 가드를 평가하는 것은 향후 과제다. 공격. 평가한 공격은 한 계열과 우리가 직접 작성한 적응 변형 하나뿐이다. 철자 이상으로 목적지를 변형하는 공격자, 또는 정당한 목적지로 보내는 내용에 데이터를 실어 유출하는 공격자는 가드가 검사하는 범위 밖에 있다. 단일 모델. 모든 요약은 한 시점의 한 모델에서 나왔다. 모델이나 시간에 따라 이 비율이 얼마나 달라지는지는 측정하지 않았다.

6 Related Work

6.1 Vetting an Individual Artifact

MCP 아티팩트 하나를 검사하는 연구는 서버가 광고하는 텍스트에서 시작한다. 도구 오염 탐지기는 설명 안에 숨겨진 지시와 이름 가로채기, 유출 엔드포인트를 읽어내고 [18], 신뢰할 수 있는 설명을 다시 생성해 그 벡터 자체를 없애기도 하며 [50], 벤치마크는 실제 서버에서 이런 공격이 얼마나 성공하는지를 정량화한다 [25, 45, 49]. 도구 설명이 그 자체로 과잉 권한을 합성하는 경로가 됨을 보인 측정도 있다 [36]. 텍스트 아래로 내려가면, 정적 분석은 신뢰할 수 없는 입력이 민감한 연산에 닿는 결함을 개별 도구 핸들러까지 추적하고 [42], 동작 감사는 도구를 샌드박스에서 실행해 관찰된 흐름을 보고하며 [44], 설정 감사는 선언된 서버의 출처와 알려진 위험 패턴을 점점한다 [29, 41]. 이 갈래에서 판정의 대상은 아티팩트 하나이고, 판정의 결론은 그 아티팩트가 악의적이거나 결함이 있는가이다. 우리가 묻는 것은 그와 독립적인 질문이다. 악의도 결함도 없이 문서화된 대로 동작하는 도구가 정상 동작으로 무엇에 닿는가.

6.2 Set-level Analysis and Information Flow

여럿이 모였을 때 비로소 성립하는 성질을 묻는 전통은 정보 흐름 제어에 있다. Denning은 허용되는 흐름을 보안 등급의 격자로 형식화해, 어떤 흐름이 허용되는지를 프로그램마다 논증하는 대신 격자 위에서 판정할 수 있게 했다 [10]. 분산 IFC는 라벨에 소유자를 기록해 서로 불신하는 주체들이 각자의 몫만 선별적으로 declassify하도록 했고 [32], 동적 taint 분석은 신뢰할 수 없는 입력에 표시를 붙여 실행 중에 전파하며 그것이 민감한 사용 지점에 닿는 순간을 잡았다 [33]. 설치된 집합을 단위로 삼는 형태도 오래되었다. Android의 앱 간 권한 유출에 대한 조합 분석은 설치된 앱 전체를 함께 분석하고, 앱이 설치·갱신·삭제될 때마다 이를 다시 검사한다 [2, 20]. 에이전트 층위에서 같은 질문은 lethal trifecta로 명명되고 toxic flow로 탐지되며 [3, 38, 46], 계획 생성과 보안 감시를 분리해 신뢰할 수 없는 입력이 제어 결정에 영향을 주지 못하게 하는 시스템 수준 IFC도 제시되었다 [47]. INTERLOCK의 Guard는 이 갈래의 능력 추적을 값의 동일성으로 근사한다.

도구 결과의 값마다 출처 클래스를 붙이고 목적지 인자가 그 값과 같은지 또는 그 안에 포함되는
 지만 보므로 값이 그대로 전달되는 흐름에 대해서는 건전하며, CaMeL처럼 질의를 프로그램으로
 옮기거나 격리된 모델을 두지 않고 보통의 도구 호출 에이전트 위에서 그대로 동작한다 [8].

6.3 Privilege Narrowing and Policy Synthesis

주체에게 필요한 만큼의 권한만 준다는 원칙은 Saltzer와 Schroeder가 정리한 여덟 설계 원칙 가
 운데 하나다 [37]. 이를 표현하는 고전적 메커니즘이 *capability* – 대상의 이름과 그에 대한 권한을
 함께 담은 위조 불가능한 토큰 – 이고 [11], 이를 프로세스 구조에 적용한 것이 권한 분리다. 특권
 데몬을 작은 감시자와 신뢰할 수 없는 입력을 파싱하는 비특권 자식으로 쪼개면 노출된 코드의 버
 그가 권한으로 이어지지 않는다 [34]. LLM 에이전트에 대해서도 같은 원칙이 적용되어, 작업에서
 정책을 합성해 호출마다 집행하거나 [40], 값에 능력을 붙여 인터프리터가 검사하게 하거나 [8],
 신뢰 수준이 다른 구성 요소를 분리하며 [7], 도달 가능성 위의 *cut*으로 능력을 가두거나 [48] 에이
 전트 루프를 가드레일로 감싸는 방법도 제시되었다 [31]. 기능을 유지한 채 권한을 줄이는 문제는
 에이전트 밖에서도 연구되어, 브라우저 확장의 과다 권한은 선언된 *manifest* 권한을 코드가 실제
 로 호출하는 API와 대조해 탐지되고 [6], 프로그램 *debloating*은 테스트 스위트를 계속 통과시켜야
 한다는 제약 아래 아티팩트를 줄인다 [15, 39]. INTERLOCK도 호출마다 검사하지만, 작업에서 정
 하는 것은 WRITE 도구의 허용 목록뿐이고 호출마다 대조하는 것은 코드에서 한 번 도출한 효과
 라벨 위의 고정된 출처 규칙이다. Progent와 달리 인자 정책을 질의마다 합성하지 않고 도구 출력
 을 읽어 정책을 갱신하는 단계도 없으므로 주입이 정책에 닿을 길이 없으며, 제3자의 아티팩트는
 고쳐 쓰지 않는다 [40].

6.4 Making a Model's Judgement Checkable

코드를 읽고 판정하는 일을 언어 모델에 맡길 때 남는 문제는 그 판정을 무엇으로 믿느냐이며,
 이에 대한 답은 크게 두 갈래다. 하나는 모델이 스스로 근거를 인용하게 하고 그 인용을 검증하
 는 쪽이다. ALCE는 인용 표식을 믿는 대신 인용된 구절이 그 문장을 실제로 함의하는지 검사해
 citation recall과 precision을 매기고, 강한 시스템조차 상당한 비율의 문장이 자신이 인용한 근거로
 뒷받침되지 않음을 보인다 [14]. 배포된 생성 검색 엔진을 사람이 문장 단위로 대조한 감사에서는
 완전히 뒷받침되는 문장이 절반 남짓에 그쳐, 유창하게 인용하는 답과 실제로 확인 가능한 답 사
 이의 간격이 수치로 남았다 [24]. 다른 하나는 모델의 출력을 결정적인 외부 검사기에 통과시키는
 쪽이다. IRIS는 정적 분석이 스스로 추론할 수 없는 *taint source*와 *sink*만 모델에서 받고 실제 경로
 계산은 CodeQL에 맡기며 [21], LIFT는 의도적으로 제약을 푼 분석기가 남긴 후보를 모델이 좁히
 되 자기 검증을 거치게 한다 [19]. Baldur는 증명 전체를 한 번에 생성하되 그것이 성립하는지는
 증명 보조기가 판정하게 하여, 건전성이 전적으로 외부 검사기에서 나오고 모델은 틀려도 되게
 만든다 [12]. INTERLOCK의 재검사는 두 번째 갈래에 속하되 검사기가 판정하는 대상이 다르다.
 우리는 모델의 결론이 참인지를 검사기에게 묻지 않고, 모델이 인용한 *file:line*이 그 결론을
 실제로 증언하는지만 결정적으로 묻는다.

6.5 Measuring the MCP and Skill Ecosystems

생태계를 대상으로 한 실증 연구는 이 층위의 위험이 어디에 얼마나 있는지를 보고한다. 공개된
 MCP 서버를 대규모로 수집한 연구들은 취약점 유형률과 분류 체계를 제시하고 [13, 28], 서버가 시
 간에 따라 어떻게 변해 가는지를 추적하며 [22], 공격 쪽에서는 MCP 고유의 공격면을 체계화하고
 실제 배포에서 재현한 시연이 축적되었다 [23, 30, 43, 52]. 인접한 에이전트 스킬 생태계도 빠르게
 축적되어 취약점 유형률과 자격증명 유출, 공급망 오염이 보고되었고 [1, 5, 16, 26, 27, 35], 페이로
 드를 여러 스킬에 분산해 아티팩트 단위 검사기를 회피하는 공격 [51]과 건전한 능력 상한을 추상
 해석으로 계산하려는 시도 [4]가 제시되었다. 다만 스킬에는 도구 목록을 협상하는 프로토콜도
 광고된 *schema*도 없어, 우리가 쓰는 측정 단위 – 설정이 선언한 서버와 그 서버가 광고한 도구
 – 에 대응하는 대상이 존재하지 않는다. 우리의 코퍼스도 이 갈래에 속하지만 모집단이 다르다.

이들이 표집하는 것은 공개된 서버나 스킴인 반면, 우리가 표집하는 것은 사용자가 실제로 작성해 커밋한 설정이다.

7 Conclusion

본 연구는 INTERLOCK을 제시했다. 도구가 무엇에 닿는지를 제공자가 쓴 글이 아니라 서버의 소스 코드에서 회수하되 모든 주장에 `file:line` 증거를 요구하고 결정적 검사기로 그 구간을 다시 읽는 Summarize와, 그 라벨을 소비하는 두 집행 규칙 Narrow와 Guard다. 공개 설정에서 가장 자주 선언되는 서버 34개, 도구 433개에 대해 라벨 주장의 72.5%가 코드로 검증되고, 구현 코드를 배포하는 서버 27개로 한정하면 90.7%다. AgentDojo에서는 도구 단위 규칙이 아무리 정확해도 targeted ASR을 4.18% 아래로 내리지 못하므로, Narrow는 사용자 요청만 보고 정한 WRITE 허용 목록 밖의 호출을 거부하고, Guard는 목적지 인자마다 값의 출처를 추적해 공격자가 쓸 수 있는 텍스트에서만 올 수 있는 값을 거부한다. 둘을 함께 두면 세 번의 실행에서 targeted ASR 1.06%에 utility under attack 61.35%로, 같은 조건의 Progent는 3.15%와 58.71%다. 남는 것은 실행당 공격 10건과 benign 과제 5건이며, 전자는 목적지가 정당하고 내용이나 선택만 악의적인 경우이고 후자는 사용자가 신뢰할 수 없는 페이지에서 목적지를 읽어 쓰라고 지시한 경우로, 값이 어디서 왔지만 보는 규칙은 이 둘을 구별하지 못한다.

References

- [1] Vivek Acharya. 2026. Skill poisoning: Attack taxonomies and defense architectures for composable agent skill ecosystems in ai-driven cyber-physical systems. *Available at SSRN 6408998* (2026).
- [2] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. Covert: Compositional analysis of android inter-app permission leakage. *IEEE transactions on Software Engineering* 41, 9 (2015), 866–886.
- [3] Luca Beurer-Kellner, Marco Milanta, and Marc Fischer. 2025. Invariant Labs exposes novel prompt injection attack vulnerabilities, “toxic flows”, in agentic systems and MCP servers. <https://invariantlabs.ai/blog/toxic-flow-analysis> Accessed 2026-09-16.
- [4] Varun Pratap Bhardwaj. 2026. Formal analysis and supply chain security for agentic AI skills. *arXiv preprint arXiv:2603.00195* (2026).
- [5] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. 2026. How your credentials are leaked by LLM agent skills: An empirical study. *arXiv preprint arXiv:2604.03070* (2026).
- [6] Xiaoyu Cheng, Zhi Wang, Wanpeng Li, and Lifei Sun. 2026. PerCheck: An AST-Based Framework for Detecting Over-Privilege in Chrome Extensions. In *International Conference on Intelligent Computing*. Springer, 13–24.
- [7] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643* (2025).
- [8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813* (2025).
- [9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. arXiv preprint arXiv:2406.13352.
- [10] Dorothy E Denning. 1976. A lattice model of secure information flow. *Commun. ACM* 19, 5 (1976), 236–243.
- [11] Jack B Dennis and Earl C Van Horn. 1966. Programming semantics for multiprogrammed computations. *Commun. ACM* 9, 3 (1966), 143–155.
- [12] Emily First, Markus N Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1229–1241.
- [13] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. 2026. Systematization of knowledge: Security and safety in the Model Context Protocol ecosystem. In *Intelligent Systems Conference*. Springer, 494–517.
- [14] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 6465–6488.
- [15] Kihong Heo, Woosuk Lee, Pardis Pashakhanloo, and Mayur Naik. 2018. Effective program debloating via reinforcement learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 380–394.

- [16] Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. 2026. Context Matters: Repository-Aware Security Analysis of the Agent Skill Ecosystem. *arXiv preprint arXiv:2603.16572* (2026).
- [17] Charoes Huang, Xin Huang, and Amin Milani Fard. 2026. Auditing MCP Servers for Over-Privileged Tool Capabilities. *arXiv preprint arXiv:2603.21641* (2026).
- [18] Gunjan Jain. 2026. *Detecting MCP Tool Poisoning and Rug-Pull Attacks in LLM Agent Architectures*. Technical Report. QCCrypton. <https://zenodo.org/records/19687220>
- [19] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 474–499. doi:10.1145/3649828
- [20] Li Li, Alexandre Bartel, Tegawendé F Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Oceau, and Patrick McDaniel. 2015. Ictta: Detecting inter-component privacy leaks in android apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 280–291.
- [21] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-assisted static analysis for detecting security vulnerabilities. In *International conference on learning representations*, Vol. 2025. 35735–35758.
- [22] Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. 2026. MCP-Evol-Bench: Benchmarking LLM Agent Performance Across Dynamic Evolutions of MCP Servers. *arXiv preprint arXiv:2607.14642* (2026).
- [23] Liwei Liu, Tianzhu Han, Zijian Liu, Zishu Dong, and Na Ruan. 2026. ShareLock: A Stealthy Multi-Tool Threshold Poisoning Attack Against MCP. *arXiv preprint arXiv:2606.27027* (2026).
- [24] Nelson F Liu, Tianyi Zhang, and Percy Liang. 2023. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 7001–7025.
- [25] Shi Liu, Xuehai Tang, Xikang Yang, Liang Lin, Biyu Zhou, Wenjie Xiao, and Wantao Liu. 2026. When the Manual Lies: A Realistic Benchmark to Evaluate MCP Poisoning Attacks for LLM Agents. In *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2090–2095.
- [26] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. 2026. "Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild. *arXiv preprint arXiv:2602.06547* (2026).
- [27] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338* (2026).
- [28] Muhammad Hamza Arshad Majeed, May Mahmoud, and Sarah Nadi. 2026. An Empirical Study of Model Context Protocol Applications. *arXiv preprint arXiv:2607.25635* (2026).
- [29] MCPAudit contributors. 2026. MCPAudit: Security auditing for MCP server configurations. <https://github.com/saagpate1/MCPAudit>. Accessed 2026-09-16; issue codes MCP013 (single-server trifecta) and MCP014 (cross-server trifecta).
- [30] Marco Milanta and Luca Beurer-Kellner. 2025. GitHub MCP exploited: Accessing private repositories via MCP. <https://invariantlabs.ai/blog/mcp-github-vulnerability> Accessed 2026-09-22.
- [31] Nandhakumar Murugan. 2026. AgentShield: A Zero-Trust Runtime Guardrail Architecture for Autonomous Multi-Agent AI Systems with Bidirectional Context Synchronization. In *Zenodo (CERN European Organization for Nuclear Research)*. doi:10.5281/zenodo.22259021
- [32] Andrew C Myers and Barbara Liskov. 1997. A decentralized model for information flow control. *ACM SIGOPS Operating Systems Review* 31, 5 (1997), 129–142.
- [33] James Newsome, Dawn Xiaodong Song, et al. 2005. Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software.. In *NDSS*, Vol. 5. 3–4.
- [34] Niels Provos, Markus Friedl, and Peter Honeyman. 2003. Preventing privilege escalation. In *12th USENIX Security Symposium (USENIX Security 03)*.
- [35] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-chain poisoning attacks against LLM coding agent skill ecosystems. *arXiv preprint arXiv:2604.03081* (2026).
- [36] Mohammadreza Rashidi. 2026. Measuring How LLM Tool Descriptions, Cross-Tool Ambiguity, and Action Chains Compose into Excessive Agency Exploits. (2026).
- [37] Jerome H Saltzer and Michael D Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308.
- [38] Chris Schneider, Kriti Faujdar, Philipp Schoenegger, and Ben Bariach. 2026. Securing Multi-Tool AI Agent Chains With Dynamic, Real-Time Compositional Policies. *arXiv preprint arXiv:2607.03423* (2026).
- [39] Hashim Sharif, Muhammad Abubakar, Ashish Gehani, and Fareed Zaffar. 2018. TRIMMER: application specialization for code debloating. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 329–339.

- [40] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703* (2025).
- [41] Snyk. 2026. agent-scan issue codes. <https://github.com/snyk/agent-scan/blob/main/docs/issue-codes.md> Accessed 2026-09-16.
- [42] Pengyu Sun, Zifeng Kang, Qishu Jin, Enhao Huang, Xin Liu, Dakun Shen, and Song Li. 2026. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers. *arXiv preprint arXiv:2605.21392* (2026).
- [43] Yiheng Sun, Linkang Du, Zhou Su, Yuntao Wang, Han Liu, Quan Zhao, and Xiaolin Niu. 2025. MSA: A Cross-MCP Privacy Attack via Memory Exfiltration of Large Language Models. In *Proceedings of the 24th Workshop on Privacy in the Electronic Society*. 177–182.
- [44] Zhuoran Tan, Run Hao, Jeremy Singer, Yutian Tang, and Christos Anagnostopoulos. 2026. SandScope: a Behavioral Audit Layer for MCP Tools in LLM Agent Supply Chains. (2026).
- [45] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2026. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 35811–35819.
- [46] Simon Willison. 2025. The lethal trifecta for AI agents: private data, untrusted content, and external communication. *Simon Willison's Weblog* (2025).
- [47] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. 2024. System-level defense against indirect prompt injection attacks: An information flow control perspective. *arXiv preprint arXiv:2409.19091* (2024).
- [48] Wujie Xiong, Rabimba Karanjai, Yang Lu, Weidong Shi, and Lei Xu. 2026. Reachability-Based Capability Confinement for LLM Agents under Indirect Prompt Injection. *arXiv preprint arXiv:2608.30041* (2026).
- [49] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. 2025. MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols. *arXiv preprint arXiv:2508.13220* (2025).
- [50] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. 2026. Trustdesc: Preventing tool poisoning in llm applications via trusted description generation. *arXiv preprint arXiv:2604.07536* (2026).
- [51] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColluSkill: Adversarial Cross-Skill Composition for Evading Agent Skill Scanners. *arXiv preprint arXiv:2608.09732* (2026).
- [52] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 138–155.